WEEK-8 SKILL

Task1 : To Detect Variable References, Expand Values, Handle Undefined Variables, Update Tokens, Support Nested Variables, Test Expansion Logic

Aim : To implement variable expansion in a shell-like environment by detecting variable references, replacing them with their values, handling undefined variables, updating tokens, supporting nested variables, and testing the expansion logic.

Step 1 — Create the C program

nano variable_expand.c

```
  GNU nano 7.2                    variable_expand.c *
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>

void expand_variables(char *input, char *output)
{
    int i = 0;
    int j = 0;

    while (input[i] != '\0')
    {
        if (input[i] == '$')
        {
            char variable[100];
            int k = 0;

            i++;

            while (isalnum((unsigned char)input[i]) ||
                   input[i] == '_')
            {
                variable[k++] = input[i++];
            }

            variable[k] = '\0';

            char *value = getenv(variable);

            if (value != NULL)
            {
                for (int x = 0; value[x] != '\0'; x++)
                {
                    output[j++] = value[x];
                }
            }
```

```
            else
            {
                printf("Undefined variable: $%s\n", variable);
            }
        }
        else
        {
            output[j++] = input[i++];
        }
    }

    output[j] = '\0';
}

int main()
{
    char input[500];
    char output[1000];

    printf("Enter text with variables: ");
    fgets(input, sizeof(input), stdin);

    input[strcspn(input, "\n")] = '\0';

    expand_variables(input, output);

    printf("Expanded text: %s\n", output);

    return 0;
}
```

Step 2 — Compile and run the program

```
alurusaranya@LAPTOP-0PA61QEN:~/OSSP$ nano variable_expand.c
alurusaranya@LAPTOP-0PA61QEN:~/OSSP$ gcc variable_expand.c -o variable_expand
alurusaranya@LAPTOP-0PA61QEN:~/OSSP$ ./variable_expand
Enter text with variables: Hello $USER
Expanded text: Hello alurusaranya
alurusaranya@LAPTOP-0PA61QEN:~/OSSP$
```

Step 3— Test an undefined variable

```
alurusaranya@LAPTOP-0PA61QEN:~/OSSP$ ./variable_expand
Enter text with variables: Hello $ABCXYZ
Undefined variable: $ABCXYZ
Expanded text: Hello
alurusaranya@LAPTOP-0PA61QEN:~/OSSP$
```

Step 4 — Test multiple variables

```
alurusaranya@LAPTOP-0PA61QEN:~/OSSP$ ./variable_expand
Enter text with variables: User: $USER, Home: $HOME
Expanded text: User: alurusaranya, Home: /home/alurusaranya
alurusaranya@LAPTOP-0PA61QEN:~/OSSP$
```

Step 5 — Test variable expansion with surrounding text

```
alurusaranya@LAPTOP-0PA61QEN:~/OSSP$ ./variable_expand
Enter text with variables: Welcome $USER to $HOME directory
Expanded text: Welcome alurusaranya to /home/alurusaranya directory
alurusaranya@LAPTOP-0PA61QEN:~/OSSP$
```

Step 6 — Test an undefined variable with a defined variable

```
alurusaranya@LAPTOP-0PA61QEN:~/OSSP$ ./variable_expand
Enter text with variables: User: $USER, Unknown: $XYZABC
Undefined variable: $XYZABC
Expanded text: User: alurusaranya, Unknown:
alurusaranya@LAPTOP-0PA61QEN:~/OSSP$
```

Step 7 — Test nested-style variable expansion

```
alurusaranya@LAPTOP-0PA61QEN:~/OSSP$ export MYNAME="$USER"
alurusaranya@LAPTOP-0PA61QEN:~/OSSP$ ./variable_expand
Enter text with variables: My name is $MYNAME
Expanded text: My name is alurusaranya
alurusaranya@LAPTOP-0PA61QEN:~/OSSP$
```

Observation:

The program successfully detected variable references such as $USER and $HOME and replaced them with their corresponding environment values using getenv(). Undefined variables were detected and reported without terminating the program. Multiple variables and variables surrounded by normal text were also tested successfully.

Result:

Variable expansion was successfully implemented and tested in the shell-like environment. The program correctly detects variable references, expands defined variables, handles undefined variables, and updates the resulting output tokens. A variable created using another environment value was also tested successfully.

Task2 : To Identify Built-ins, Create Dispatch Table, Execute In-Process Commands, Handle Invalid Commands, Maintain State, Test Dispatch Logic

Aim : To identify shell built-in commands, create a dispatch table for those commands, execute built-in commands within the shell process, handle invalid commands, maintain shell state, and test the command-dispatch logic.

Step 1 — Create the built-in command program

builtin_dispatch.c

```
  GNU nano 7.2                          builtin_dispatch.c *
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>

void builtin_pwd()
{
    char cwd[1024];

    if (getcwd(cwd, sizeof(cwd)) != NULL)
    {
        printf("%s\n", cwd);
    }
    else
    {
        perror("pwd");
    }
}

void builtin_cd(char *path)
{
    if (path == NULL)
    {
        printf("cd: missing argument\n");
        return;
    }

    if (chdir(path) != 0)
    {
        perror("cd");
    }
}

void builtin_exit()
{
    printf("Exiting shell...\n");
    exit(0);
}
```

```
  GNU nano 7.2                          builtin_dispatch.c *
void execute_builtin(char *command, char *argument)
{
    if (strcmp(command, "pwd") == 0)
    {
        builtin_pwd();
    }
    else if (strcmp(command, "cd") == 0)
    {
        builtin_cd(argument);
    }
    else if (strcmp(command, "exit") == 0)
    {
        builtin_exit();
    }
    else
    {
        printf("Invalid command: %s\n", command);
    }
}

int main()
{
    char command[100];
    char argument[200];

    while (1)
    {
        printf("ossp> ");

        scanf("%99s", command);

        if (strcmp(command, "cd") == 0)
        {
            scanf("%199s", argument);
            execute_builtin(command, argument);
        }
        else
        {
            execute_builtin(command, NULL);
        }
    }

    return 0;
}
```

Step 2 — Compile the program

```
alurusaranya@LAPTOP-0PA61QEN:~/OSSP$ nano builtin_dispatch.c
alurusaranya@LAPTOP-0PA61QEN:~/OSSP$ gcc builtin_dispatch.c -o builtin_dispatch
alurusaranya@LAPTOP-0PA61QEN:~/OSSP$ ./builtin_dispatch
ossp> pwd
/home/alurusaranya/OSSP
ossp> cd /tmp
ossp> pwd
/tmp
ossp> hello
Invalid command: hello
ossp> exit
Exiting shell...
alurusaranya@LAPTOP-0PA61QEN:~/OSSP$
```

Observation:

The shell successfully identified built-in commands such as pwd, cd, and exit and dispatched them to their respective functions. The pwd command displayed the current working directory, while cd changed the shell's current directory using chdir(). Invalid commands were detected and reported with an appropriate error message. The exit command successfully terminated the shell.

Result:

Built-in command identification, command dispatch, in-process execution, invalid-command handling, shell-state maintenance, and dispatch logic were successfully implemented and tested.